Fetch Me If You Can: Evaluating CPU Cache Prefetching and Its Reliability on High Latency Memory

Fabian Mahling
Hasso Plattner Institute, University of Potsdam
Potsdam, Germany
fabian.mahling@student.hpi.de

Marcel Weisgut
Hasso Plattner Institute, University of Potsdam
Potsdam, Germany
marcel.weisgut@hpi.de

Tilmann Rabl
Hasso Plattner Institute, University of Potsdam
Potsdam, Germany
tilmann.rabl@hpi.de

Abstract

Memory can be located close to a CPU, at remote sockets, or on devices connected via interconnects such as CXL or NVLink. A larger distance between memory and a core accessing the memory usually results in higher access latency. Software prefetching algorithms claim to hide memory access latencies by moving data to the CPU cache before a core accesses the data. In this work, we analyze to what extent software prefetching can hide increased memory access latencies. We evaluate these on seven systems, each offering different memory technologies and access latencies. We show that prefetching can increase performance by up to 2.6× and 2.8× for B$^+$-Tree and binary search workloads. We find that CPU fill buffers, which track L1 cache misses, and a workload's memory intensity dictate how much access latency can be hidden. CPUs implement prefetches differently. We introduce microbenchmarks that identify concrete target cache and eviction strategies for different prefetch localities across x86 and ARM architectures. When the fill buffers are full, CPUs either drop prefetches or halt until all can be executed. We refer to these behaviors as weak and strong prefetching reliability. We introduce microbenchmarks identifying a CPU's reliability. When prefetching 8 KiB B$^+$-Tree nodes, weak reliability achieves a speedup of 2× while strong reliability degrades performance with a slowdown of 2.5× for lookup workloads.

CCS Concepts

• **Information systems** → **Data access methods**; • **Hardware** → **Memory and dense storage**.

ACM Reference Format:
Fabian Mahling, Marcel Weisgut, and Tilmann Rabl. 2025. Fetch Me If You Can: Evaluating CPU Cache Prefetching and Its Reliability on High Latency Memory. In *21st International Workshop on Data Management on New Hardware (DaMoN '25), June 22–27, 2025, Berlin, Germany*. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3736227.3736231

1 Introduction

Modern in-memory database systems store data primarily in DRAM [10, 12, 16, 32]. When accessing data in a non-sequential or random manner, e.g., by probing a hash table or traversing a B$^+$-Tree, the required data is often not in the CPU's cache [15]. This leads to CPU stalls as the CPU needs to load the data from memory. Memory latency can easily dominate the runtime in such scenarios [15].

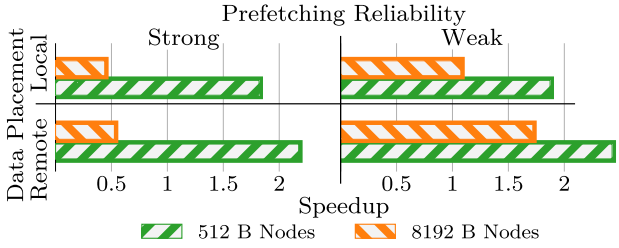

Figure 1: B$^+$-Tree lookup speedups when prefetching entire nodes from local and remote memory on CPUs with strong (Xeon-2) and weak (EPYC-3) prefetching reliability.

Recent work proposed prefetching data to the CPU's cache before accessing it by interleaving request streams to reduce stalls [13, 30]. This requires batched requests against the data structures. Batched requests can occur when processing queries concurrently or in parallelizable algorithms, e.g., hash joins [13, 31]. Multiple algorithms for instruction stream interleaving exist, such as asynchronous memory access chaining, group prefetching, or coroutine-based prefetching [4, 17, 30]. Coroutines are the most promising approach, achieving good performance at low implementation effort [15].

Memory is a heterogeneous resource. Multiple forms of DRAM exist, such as double data rate (DDR), high bandwidth memory (HBM), graphic DDR (GDDR), and low-power DDR (LPDDR). These can be placed directly onto the CPU package, distributed among CPU and memory sockets, or on devices connected via interconnects such as CXL or NVLink [8, 21, 33]. Memory access latency varies among server architectures, depending on the distance between CPU and memory [21]. There is little research on the impact of increased memory access latencies on prefetching [9, 13, 15].

Prefetch instructions do not specify concrete hardware implementations but are hints [20, 22]. CPUs implement them differently, leading to different target cache levels and eviction strategies [20]. CPUs react differently to excessive prefetching. Some drop prefetches if hardware resources get overloaded and others stall until prefetches can be issued [20]. We refer to these behaviors as weak and strong prefetching reliability. CPU documentation often lacks this level of detail. While existing work investigates prefetching characteristics of specific CPUs based on existing documentation, we provide tools to quantify such characteristics across multiple CPUs.

DaMoN '25, Berlin, Germany

ACM ISBN 979-8-4007-1940-0/25/06
https://doi.org/10.1145/3736227.3736231

Contributions. Our main contributions are the following:

- Open-source microbenchmarks[1] that identify CPU prefetching characteristics. We use these to systemically characterize seven different ARM and x86 CPUs (Section 3).
- A prefetching performance evaluation using B$^+$-Tree and binary search workloads. We place data in CPU-local memory and memory with higher latency, including remote socket memory, GPU HBM, and CPU HBM (Section 4).
- A performance analysis of strong and weak prefetching reliability with B$^+$-Tree operations and binary search workloads (Section 4).
- Guidelines on when and how to use prefetching (Section 5).

Our experiments show that prefetching reliability can significantly impact B$^+$-Tree lookup performance as illustrated in Figure 1. On 8 KiB B$^+$-Tree nodes, with data placed on high-latency memory, CPUs with weak reliability show speedups between 1.1× and 2×, while strong reliability leads to speedups from 0.4× to 1×. Prefetching in binary search achieves speedups from 1.4× to 2.1× on low-latency memory and 1.9× and 2.8× on high-latency memory.

2 Software Prefetching

Prefetch instructions load data into caches, reducing cache miss penalties and memory stalls when the data is accessed. When executing a prefetch, the target address is first translated into a physical address. Translation lookaside buffer (TLB) misses are handled synchronously and add additional latency [20]. After the address translation, the CPU attempts to load the data from L1 [20]. On an L1 miss, the CPU inserts the prefetching request into a buffer, which is then handled asynchronously. Intel, AMD, and Fujitsu call this buffer *line fill buffer (LFB)*, *miss address buffer (MAB)*, and *move in buffer (MIB)*, respectively [20, 26]. We refer to this buffer as *fill buffer (FB)*. The FB is located between the L1 and L2 caches. For every main memory, L2, and L3 request, the CPU allocates an FB slot to track the request [20]. Prefetch requests are either dropped if the FB is full or the CPU stalls until a slot is available. Fujitsu refers to this as weak or strong prefetch reliableness, respectively [26]. We refer to it as *prefetching reliability*.

Prefetch instructions are treated as hints. As such, the CPU might ignore these instructions [6, 20]. Prefetch instructions affect performance but not the program logic [14]. Software prefetches positively impact performance when issued well-timed ahead of the actual load. If issued too close to the access, the data will not be copied to the cache by then; if issued too far away, the data will be evicted when performing the load.

Clang and *gcc* support the `__builtin_prefetch()` function. It takes a pointer, an integer indicating read or write access, and a locality hint as arguments. On x86, four different prefetch instructions for reads exist based on the different locality hints, `NTA`, `T0`, `T1`, and `T2`. `T0`, `T1`, and `T2` generally prefetch data into the L1, L2, and L3 caches, respectively, as displayed in Figure 2. NTA suggests non-temporal data, signaling to the CPU that data is accessed only once and can be evicted early from caches [20]. ARM offers six possible instructions based on the two policies `KEEP` and `STREAM` and the three targets `L1`, `L2`, and `L3` [20, 24]. `STREAM` suggests non-temporal data, similar to NTA. `KEEP` suggests that the data can be kept in cache [24]. These different instructions are mapped to concrete implementations on the CPU side. A CPU does not necessarily implement a specialized instruction for each hint. Intel treats `T1` and `T2` the same on their *Xeon Scalable* family [7]. AMD claims to either map `T1` and `T2` to `T0` or to map all prefetches to a single `PREFETCH` (*3DNow!*) instruction [1, 2]. Data movement can also be triggered by regular memory loads and hardware prefetches. Software is responsible for initiating both memory loads and software prefetches, whose access patterns in turn impact and trigger hardware prefetches [20]. Hardware prefetchers can be located at different levels of the caching hierarchy, most prominently at the L2 cache [19]. Here, the FB between L1 and L2 does not limit the hardware prefetchers. Instead, higher-level queues and buffers restrict the maximum number of outstanding requests [19, 26, 34]. Each time a cache line cannot be found in the core-local caches, the caching/home agent (CHA) is queried for the data location [18]. The path for the request, and the returning data, can include multiple interconnects, CHAs and finally, the memory controller (MC) connected to the memory. While all these components influence performance and latency, recent work has identified the FB to be the main bottleneck for latency-bound workloads [19, 20].

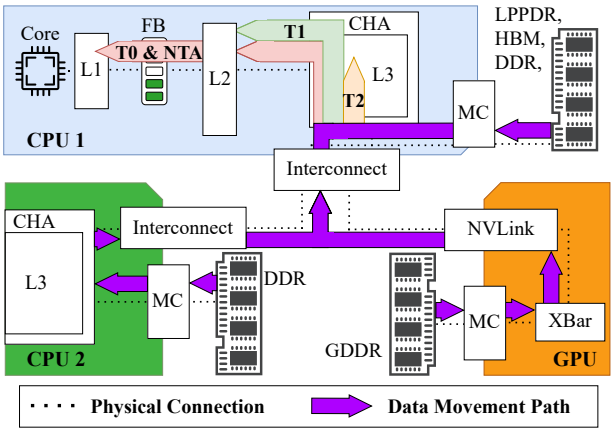

Figure 2: Combined and simplified schematic of the different memory systems used in this work. Based on [20, 34, 37].

2.1 Analyzed Systems and Setup

We run benchmarks on seven systems, described in Table 1. For the database workloads (see Section 4), the remote setting refers to data placed in remote socket memory for the AMD EPYC and Intel Xeon systems, e.g., the access path between CPU2 and CPU1 in Figure 2. Table 1 lists the resulting latencies. A64FX's remote setting refers to data placed in memory of distant core groups [26]. Grace refers to an NVIDIA GH200 chip, where a Grace CPU and Hopper GPU are connected via NVLINK-C2C, e.g., the access path between GPU and CPU1 in Figure 2, containing the Crossbar (XBar) interconnect [28, 36, 37]. Here, data is placed in GPU memory in the remote setting. Systems EPYC-2, EPYC-3, and Xeon-3 run Linux version 5.4. Xeon-E5 and Xeon-3 run Linux 5.15. ARM1 and ARM2 run Linux 4.18 and 6.2. We compile with gcc 12. Transparent huge pages (THP) are configured with a size of 2 MiB. The cache line size is 64 B on all systems except ARM1, which uses 256 B. We repeat measurements ten times and report the median.

[1] Source code: https://github.com/hpides/prefetching

Table 1: Specifications of analyzed systems. * marks verified FB sizes based on [3, 25, 29].

ID	CPUs	Memory	Latency [ns]		FB
			Local	Remote	size
EPYC-2	2× 7742	DDR4	153	277	21
EPYC-3	2× 7413	DDR4	83	272	24*
Xeon-E5	2× E5-2689 v4	DDR4	80	130	10*
Xeon-2	2× 5220S	DDR4	76	156	12
Xeon-3	2× 8352Y	DDR4	83	134	12
A64FX	1× A64FX	HBM2	215	307	12*
Grace	1× Grace CPU	LPDDR5X HBM3	181	760	16*

3 Identifying CPU Prefetching Characteristics

We run two microbenchmarks to understand the underlying prefetching hardware characteristics. We determine the hardware implementations of the different localities using the first microbenchmark and the prefetching reliability using the second.

3.1 Different Prefetch Localities

Prefetch instructions specify target localities across all instruction sets. However, the concrete prefetch effect depends on the actual hardware implementation, as described in Section 1. CPUs might ignore prefetching instructions or move data into different cache hierarchies, depending on the specified localities.

Implementation. CPUs offer no direct way of extracting the cache location corresponding to a specific memory address. However, we can measure access latencies when resolving addresses. These latencies can then be compared to reference latencies measured for cache lines with known locations in the cache hierarchy. In this benchmark, we place data into the cache or leave it in memory and compare these latencies with the latencies of prefetched cache lines. To accomplish this, we create a value array containing characters and an offset array containing 64-bit unsigned integers. We use the offsets as indices into the value array. The offsets are generated using a uniform random distribution. We place the cache lines into different cache levels in a first pass over the offset array. We do this either by doing nothing (*Upper* in Figure 3), summing up the referenced values (*Lower*), or prefetching with specific localities (*NTA*, *T0*, *T1*, *T2*). When doing nothing with the offsets, corresponding cache lines should reside in memory. Accessing the referenced values moves the corresponding cache lines to the L1 cache.

In a second pass over the offset array, we measure the time it takes to sum up all referenced values. We then compare the latencies of the three different approaches. This allows us to approximate the location within the cache hierarchy for the prefetched approach.

Some CPUs drop prefetches when too many are issued. We prevent drops from happening by *binding* prefetches to a blocking memory load. For this, we increment the offset used as index into the value array by a dependency value. This dependency value is always 0. To make the index increment blocking, we dereference a random pointer, pointing to a value 0, and add that value onto the offset per prefetch, as shown in Listing 1. This effectively limits in-flight memory requests to two: the binding memory load and the prefetch.

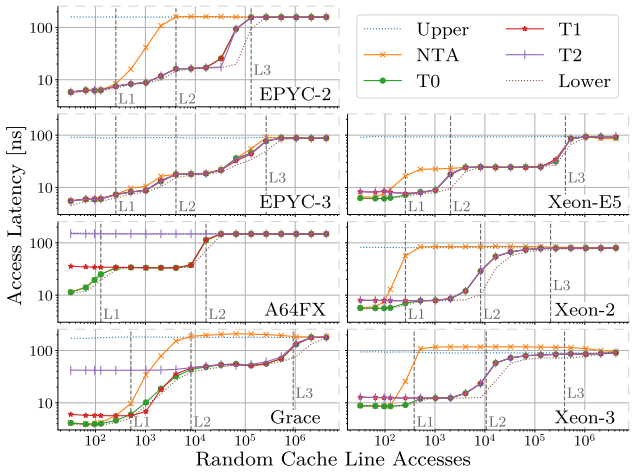

Figure 3: Access latencies after prefetching cache lines. More data is loaded with larger arrays, triggering cache evictions.

```
int dependency = 0;
for (int i = 0; i < number_prefetches; ++i){
    prefetch(addressof(values[offsets[i] + dependency]));
    if (bind_prefetch_to_memory_load){
        dependency += zeroes[random_offsets[i] + dependency];
    }
}
```

Listing 1: Binding prefetches to memory loads.

Configuration. We execute these measurements with a 512 MiB large value array, represented as a vector of characters. We activate THP and use all different locality hints `NTA`, `T0`, `T1`, and `T2`. We vary the size of the offset array between 32 and 7 M entries. As each entry defines one random cache line access, this results in data movement of 2 KiB to 430 MiB with a respective cache line size of 64 B. We repeat the measurement until at least 10 M accesses are measured per setting.

Evaluation. Figure 3 shows the measured access latencies. Vertical lines indicate the sizes of the L1, L2, and L3 caches in number of moved cache lines. Access latency plateaus appear for the L1, L2, and L3 caches and memory accesses for all systems except A64FX, Xeon-2, and Xeon-3. For A64FX the L3 plateau does not appear as it lacks an L3 cache. The two Intel CPUs feature non-inclusive last-level victim caches. Here, only a small number of L2-evicted cache lines appear in L3, resulting in no noticeable plateaus for L3. Latencies increase between the plateaus with increasing number of accesses. The increases in latency represent evictions to higher-level caches or main memory. The only system that implements all prefetching localities differently is Grace: both `NTA` and `T0` fetch data into L1. However, the `NTA`-prefetched cache lines are evicted directly to memory instead of L2 and later L3. This aligns with non-temporal data supposedly not being accessed twice. Thus, there is no need to keep it cached. `T1` and `T2` move data to L2 and L3, respectively. Overlapping lines in Figure 3 indicate identical locality implementations of a CPU. This is the case for all prefetches on the AMD systems, except for `NTA`, which displays early cache evictions. Intel CPUs also share the same implementations for `T1` and `T2`.

Summary. Different CPU architectures implement prefetching instructions differently, even among CPUs of the same manufacturer. AMD's prefetch is oblivious to the defined locality hint on the two benchmarked CPUs (Zen 2 and Zen 3) and only changes the cache eviction behavior when `NTA` is used. Intel implements specific prefetching behavior based on the locality but also merges `T1` and `T2` on our used CPUs.

3.2 Reliability of Software Prefetching

A CPU's FB slots become sparse when issuing large amounts of prefetches [20]. As a prefetch instruction is merely a hint, the CPU might drop the prefetch request [20]. We describe the behavior as *weak prefetching reliability* if prefetch requests are dropped. A CPU has *strong prefetching reliability* if it executes prefetch requests independent of the availability of resources, i.e., by stalling until allocated resources are free again. We determine this property using a microbenchmark. The prefetching reliability property of CPUs is rarely specified by vendors, e.g., only for A64FX in this work's used CPUs, for which the reliability is configurable [26]. We use both reliability settings for A64FX in the experiments.

Implementation. We again create a value and offset array as described in Section 3.1. Using multiple offsets at once, we perform batches of random accesses into the value array. We divide the processing of a batch into two phases and measure the duration of each phase. First, we prefetch all values using `NTA` locality. Second, we perform the actual accesses by summing up the values until we measure 10 M accesses. We guarantee sequential execution of the batches by adding the sum of each batch's referenced values to the offsets of the next batch. This does not change the offsets since the values are zero-initialized. We also use this data dependency during the access phase, causing each memory access to start only after the prior is finished. This way, cache misses are more evident in the recorded latencies as the CPU cannot load multiple addresses in parallel.

Configuration. We vary the batch size from 1 to 32. The value array's size is configured to 512 MiB, and the benchmarks are executed single-threaded. We enable transparent huge pages (via the `madvise` system call) to avoid TLB misses.

Evaluation. Figure 4 shows the results. The behavior is consistent across all systems for small batch sizes: prefetching takes little time, and access runtimes are large, as the prefetched cache lines are still in flight. When the batch size exceeds a certain threshold, we observe two behaviors. For both EPYC systems, Grace, and A64FX-Weak, the prefetching runtimes stay low, and the access runtimes increase more steeply with a growing batch size. For the other systems, the prefetching runtimes show a steep increase followed by a steady growth with larger batch sizes. The access runtimes remain constant or grow slightly. The threshold is nearly identical to the FB size. For the CPUs with known FB sizes (EPYC-3: 24 [20], Xeon-E5: 10 [20], A64FX: 12 [26], Grace: 16 [3]), the threshold is within the FB size ± 1: with our measurements, we identify 23 on EPYC-3, 10 on Xeon-E5, 13 on A64FX, and 16 on Grace.

The measured prefetching latencies increase drastically only when CPUs stall until new FB slots are available. Thus, the systems that display such behavior have strong prefetching reliability, while the others have weak reliability. The access runtimes for the CPUs

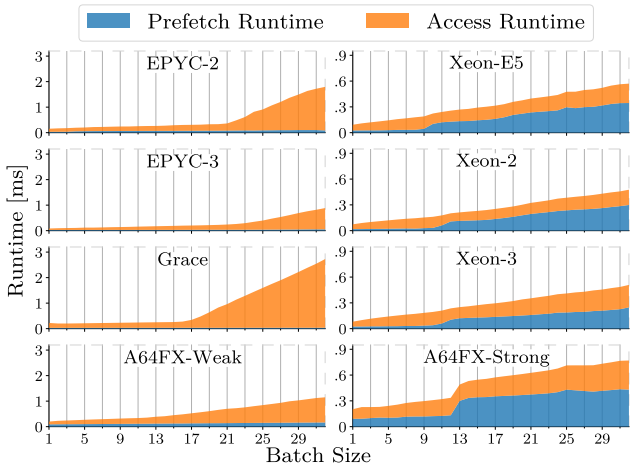

Figure 4: Absolute runtime spent on prefetching and accessing respectively in reliability benchmark.

with weak reliability see a steady increase after the batch size exceeds the FB size due to the binding of the memory loads in the access phase. Before the threshold, all requests are in flight after the prefetches are done. After the threshold, some prefetches are dropped, meaning they are not yet in flight when the actual access happens. This means that the whole memory access latency is paid.

Summary. The analyzed AMD CPUs and Grace exhibit weak prefetching reliability, while the Intel CPUs have strong reliability. We can approximate the FB size by identifying the threshold, after which either weak or strong prefetching reliability behavior is visible. We list the FB sizes in Table 1.

4 Prefetching Impact on Database Operations

Software prefetching is commonly used to improve the performance of database operations [4, 5, 13, 22, 30, 31]. We analyze the effects of remote data placement on software prefetching with lookup workloads on differently configured and implemented B$^+$-Trees in Section 4.1 and binary searches in Section 4.2.

4.1 B$^+$-Tree

We analyze the B$^+$-Tree with optimistic lock coupling (OLC) [23].

Implementation. The C++ B$^+$-Tree implementations used in this section are based initially on public implementations of the OLC B$^+$-Tree[2] [23, 35]. This code base was then extended by Kühn et al. and kindly made available to us [20]. Kühn et al. introduced coroutines to enable prefetching based on similar previous work [15, 30]. Coroutines are suspendable and resumable functions. Each lookup is wrapped inside a coroutine. When we look up multiple keys, we create multiple corresponding coroutines. During B$^+$-Tree traversal, we prefetch the node we are about to access and then suspend the current coroutine. We resume another coroutine instead of directly accessing the prefetched node. The memory loads can then finish asynchronously. When we resume the original coroutine, the prefetched node is cached. This causes fewer memory stalls, effectively hiding the memory access latencies and improving

[2]Base B$^+$-Tree source code: https://github.com/wangziqi2016/index-microbench

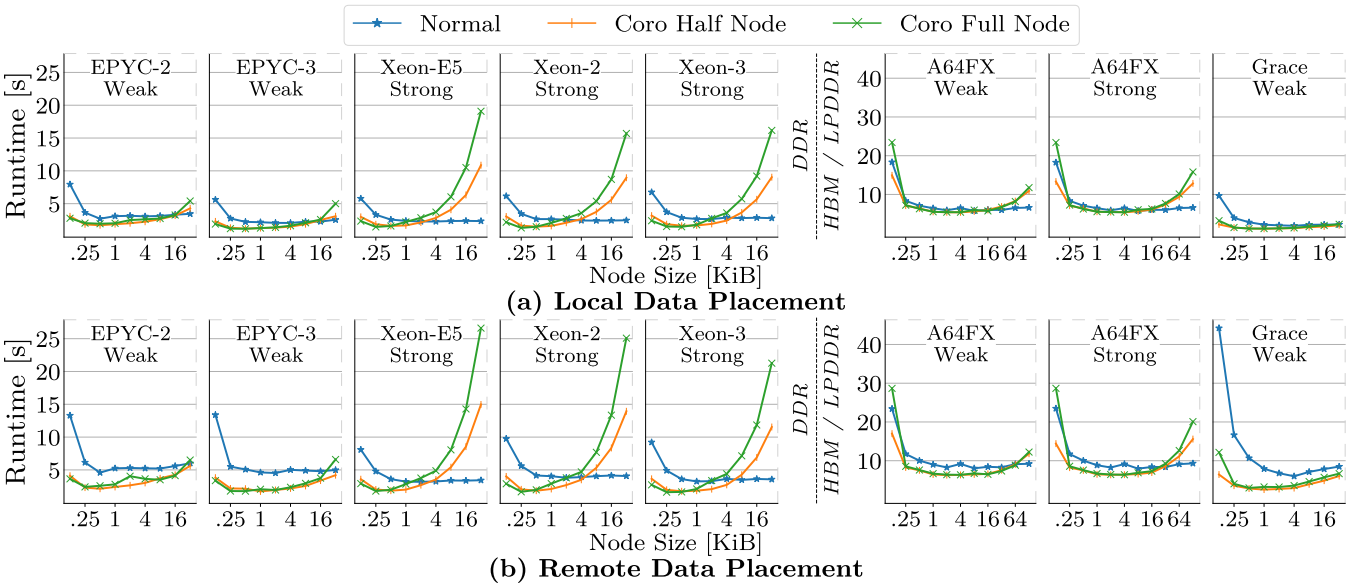

Figure 5: B⁺-Tree runtimes given different node sizes and implementations. Data is placed locally in (a) and remotely in (b).

performance. Kühn et al. implement this coroutine-based approach at two granularities. The whole node is prefetched upon access in *Coro Full Node*. *Coro Half Node* first prefetches only the header and, subsequently, the keys. In the Leaf node, the final value is also prefetched before access. Kühn et al. compare the OLC B⁺-Tree with Coro Full Node [20]. We extend the implementations with a continuous allocator, allowing us to use THP and to pin the B⁺-Trees to specific NUMA nodes using the `mbind` system call.

Configuration. We use powers of two as node sizes, starting with 128 B, and going up to 32 KiB. On A64FX, we increase the node size up to 128 KiB to better demonstrate the runtime trend. We insert 50 M key-value pairs, both of data type `uint64_t`. We run the benchmarks single-threaded and lookup 5 M elements five times. We place data remotely and use 20 coroutines. While 20 coroutines represent a best-effort configuration, more optimal setups may exist depending on the system and node size. The observed performance trends remain consistent across different coroutine configurations.

Evaluation. We present the runtimes of B⁺-Tree implementations at varying B⁺-Tree node sizes with local and remote data placement in Figure 5. For node sizes up to 1 KiB, Coro Half and Full Node offer similar runtimes across all systems. They achieve maximum speedups between 1.1× and 1.9× with local data placement and between 1.3× and 2.6× with remote data placement compared to Normal, i.e., without prefetching. For larger node sizes, the Coro runtimes increase rapidly on systems with strong prefetching reliability and slowly on systems with weak reliability. Runtimes stay relatively constant for Normal. This observation confirms that prefetching reliability is essential in this workload.

Prefetching an 8 KiB node requires moving 128 cache lines and allocating 128 respective FB slots. This exceeds the FB size of all used systems. CPUs with strong prefetching reliability stall to load the whole node before continuing with other coroutines, defeating the purpose of prefetching. This exact behavior is reflected in the `L1D_PEND_MISS.FB_FULL` performance counter on Xeon-3. When switching from 512 B to 8 KiB nodes, the counters increase from 332 to 2712 for Coro Full Node with remote data placement. Only the leaf node seems to be uncached, as 119 cache misses are reported for Coro Full Node, which is close to the size of one node (128 cache lines). Normal, on the other hand, shows 15 cache misses. This hints at the main issue: we prefetch more than we need and do not access every prefetched cache line. The CPUs with weak prefetching reliability omit this issue by dropping any prefetches that encounter a full FB. Therefore, additional latency imposed by prefetching is minimal, while part of the node is prefetched. This prefetched part includes the node header, as it is located in the first cache line of a node. As it is always accessed, we can hide some latencies and see performance improvements on EPYC-2, EPYC-3, and Grace. Runtime differences for A64FX between strong and weak reliability are only visible once the node size exceeds 16 KiB. We attribute this to the increased cache line size of 256 B vs. 64 B on all other systems. This inherently reduces the pressure on the FB as only a fourth of the requests are required to load a node. Furthermore, HBM2 offers higher bandwidth compared to DDR4 DRAM, reducing the required loading time of the nodes. Therefore, the impact of prefetching reliability is visible only on larger nodes compared to the other systems.

Node sizes between 256 B and 512 B appear optimal for the systems equipped with DDR4-attached DRAM. For A64FX, equipped with HBM2, a node size of 2 KiB seems optimal. With local data placement, Grace has the best performance at 1 KiB. The performance differences are minimal here, likely due to the large L3 cache (111 MiB). Given local data placement, 4 KiB nodes perform the best with Normal, and 1 KiB nodes with Coro Half Node.

Summary. Prefetching reliability is vital in workloads where cache lines are prefetched speculatively, and all FB slots are occupied. Here, CPUs with strong prefetching reliability will stall until all prefetches occupy a respective slot. CPUs with weak reliability drop prefetches, imposing minimal overhead.

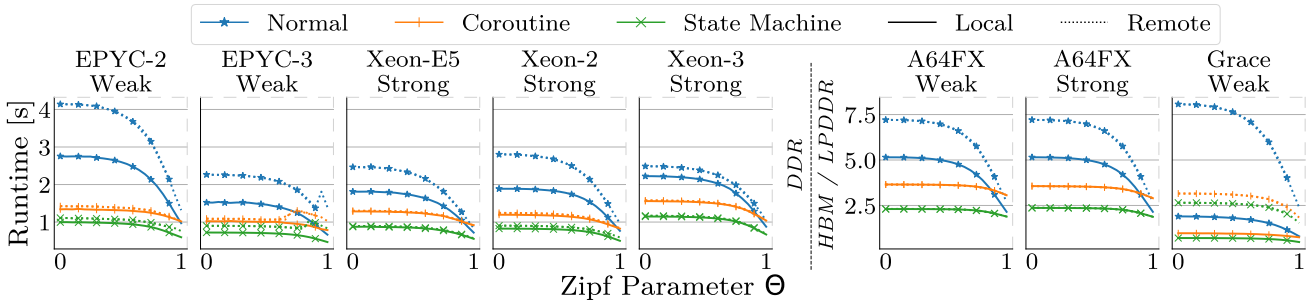

Figure 6: Binary search runtimes given different data placement, implementations, and Zipf parameters.

4.2 Binary Search

Binary search can be used on sorted arrays to find elements in logarithmic runtime. It is, for example, used during B$^+$-Tree node traversal to find the next node pointer.

Implementation. We base our implementations on three publicly available[3] binary search variants [27]. The implementations include a standard binary search implementation, a coroutine-based implementation, and a state machine-based implementation. We refer to them as Normal, Coroutine, and State Machine. The Coroutine and State Machine implementations are based on the same idea as the B$^+$-Tree implementations above. We execute multiple binary searches in parallel and jump between the searches to avoid cache misses, prefetching the elements to be accessed before jumping to the execution of other searches. Early approaches to implementing this behavior include hand-crafted state machines [4, 5]. While these state machines can be very performant, their implementation is more complex: the State Machine implementation takes 84 lines of code, while the Coroutine implementation takes 20.

We extend the Coroutine and State Machine approaches with a `reliability` parameter, which turns prefetching reliability to weak or strong on A64FX. We use the same memory allocation strategy as for the B$^+$-Tree, allowing us to place the sorted array on arbitrary non-uniform memory access (NUMA) nodes.

Configuration. We enable THP and set the number of parallel instruction streams for Coroutine and State Machine to 30, yielding good performance across all systems. We set the array size to 100 M elements. This results in a data size of about 380 MiB, as the array contains 4 B integer values. The array is filled with sequentially increasing integers, starting from zero. We measure runtimes for differently configured Zipf lookup distributions [11]. The distributions can be configured using the Zipf parameter $\theta \in (0, 1)$, where θ close to 0 yields a uniform-like distribution [11]. The closer θ gets to 1, the larger the data skew [11]. We run the binary search with θ from 0 to 1 in 0.05 steps, replacing 0 with 0.0001 and 1 with 0.9999. We prefetch using the locality hint `T0`.

Evaluation. We report runtimes of the different binary search implementations using Zipf distributions in Figure 6. On EPYC-2 at $\theta = 0.9999$, this distribution causes, on average, six DRAM loads per lookup vs. 24 given the prior, uniform distribution. The reduced number of DRAM accesses and increased number of cache hits reduce the number of backend stalls. Normal results in 211 backend stalls with a Zipf distribution at $\theta = 0.9999$ and 545 with a uniform distribution on locally placed data. This limits the possible speedup, which can be calculated as

$$\text{speedup} = \frac{T_{Normal}}{T_{Interleaved}} = \frac{T_{Compute} + T_{Stall}}{T_{Compute} + T_{Switch} + \frac{T_{Remaining\ Stalls}}{G}} \text{ [31].}$$

$T_{Compute}$ refers to the time occupied by the underlying computation and T_{Stall} to time spent in stalled cycles. T_{Switch} refers to time added by coroutine or state machine-based instruction stream jumping, $T_{Remaining\ Stalls}$ to the time spent in the remaining stalls after the instruction stream interleaving is introduced, and G to the number of parallel instruction streams [31]. If T_{Stall} decreases, the maximum achievable speedup decreases. In our cases, Coroutine and State Machine only reduce the number of stalled backend cycles to 202 and 166 on EPYC-2 with local data placement and $\theta = 0.9999$. Due to the high amount of required instructions, Coroutine does not achieve any speedup here. As T_{Switch} and $T_{Remaining\ Stalls}$ are lower for State Machine, it still achieves a speedup of about 1.7×.

For Normal, the runtime decreases across all systems with increasing θ as fewer cache misses occur. This increase is also present in Coroutine's and State Machine's runtimes. It is, however, much smaller on these implementations, as prefetching effectively hides cache misses even in uniform distributions. This means that with increasing θ, T_{Stall} decreases, which decreases achievable speedups as visible in Figure 6. For high θ, T_{Stall} gets even smaller than the overhead imposed by Coroutine for local data placement. Here, using the prefetching algorithms leads to performance degradation. However, this only is the case for θ larger than roughly 0.9 and, at worst, leads to degradation of about 30% on A64FX.

EPYC-3's runtimes increase for remotely placed data across all implementations given high θ, e.g., for Normal at $\theta = 0.95$. Perf analysis shows that, counterintuitively, cache misses increase at these points. We believe that data is evicted from the cache more aggressively for these distributions if the original memory resides on remote sockets. This behavior is likely caused by system-specific caching intricacies as we only observe it on this system.

Summary. Speedups for locally placed data vary between 1.4× and 2.1×. The prefetching implementations can hide increased memory access latencies, leading to speedups between 1.9× and 2.8× for remotely placed data. Enough FB slots are required to hide additional latencies, else the increased latency cannot be hidden as visible in Figure 6 for EPYC-3 and Grace.

[3]Source code: https://github.com/GorNishanov/await/tree/master/2018_CppCon/src

5 Discussion

Out of the seven thoroughly analyzed CPUs, only two share the same prefetching implementations given different localities. However, most concrete implementations differ only slightly. Most notably, `T0` always prefetches to the L1 cache and does not lead to quicker cache evictions of prefetched data. `NTA` also always moves data to the L1 cache. Data is marked for quicker eviction or directly evicted to memory on six out of seven CPUs. The other prefetch localities show larger differences, i.e., `T2` prefetches to the L1 cache on the AMD CPUs but to L3 cache on the Grace CPU. Using `T0` or `NTA` yields consistent behavior on most CPUs. When using `T1` or `T2`, one should expect different CPU behavior.

We categorize the prefetching reliability into strong and weak using a microbenchmark. The reliability is strong on three CPUs, weak for three CPUs, and configurable on the A64FX CPU. Kwon [19, p. 13] claims that strong prefetching reliability leads to better performance in memory-intensive programs, as they usually contain few other instructions that the CPU can execute. Switching between strong and weak prefetching reliability generally shows no large impact on database workloads. We have identified a scenario in which prefetching reliability significantly impacts performance. In this scenario, the number of issued prefetches must exceed the number of available FB slots, and only part of the prefetched data is accessed (speculative prefetching). Strong prefetching reliability leads to CPU stalls that later have no positive performance impact, as the prefetched data might never be accessed. With weak prefetching reliability, only those parts of the requested data are prefetched for which the CPU has free resources, which leads to performance gains if these are later accessed. One such workload is prefetching large B^+-Tree nodes (see Section 4.1). We suggest limiting the number of speculatively prefetched cache lines, e.g., to the FB size, to avoid negative performance impact. As systems with weak prefetching reliability only prefetch the first couple of cache lines, sorting the prefetched cache lines by access probability can improve performance. In the B^+-Tree workload, cache lines containing the header and the middle key are accessed with high probability. Ideally, all CPUs support configurable prefetching reliability for such scenarios. As this is not the case, we argue that strong reliability is preferable: the guarantee that prefetched data is loaded makes programming with prefetching simpler and more robust, as the behavior is consistent with different levels of memory access intensity. Performance-wise, the prefetching reliability does not significantly impact the other workloads discussed in this work. Additionally, the differences between strong and weak prefetching reliability will likely get smaller as FB sizes seem to grow with newer CPUs, making it harder to fill them up [20].

We evaluate prefetching algorithms on various CPUs and memory technologies. Prefetching algorithms can hide increased memory access latencies if enough FB slots are available, as visible in Figure 6. The application's memory intensity and access latency determine the required number of FB slots. New technologies such as CXL promise memory extension and high bandwidth but add access latency [33]. CPUs, on the other hand, are evolving too, showing increasing FB sizes over the last years [20]. Both trends increase the importance of memory level parallelism (MLP), for which prefetching is an adequate enabler, as displayed in this work.

Integrating prefetching into database workloads is an intricate task. Coroutines are popularly used in prefetching approaches, as they do not require much implementation effort and impose little overhead [13, 27, 31]. While their implementation is more straightforward than handcrafted approaches, we have found that the overhead they impose is noticeable as shown in Section 4.2. The impact of this overhead is negligible with high memory access, making coroutines well-suited.

6 Related Work

This work is the first to evaluate the impact of remote socket data placement and higher memory access latencies on the performance of software prefetching algorithms across multiple CPUs, interconnects, and DRAM forms. We introduce microbenchmarks designed to evaluate different prefetch localities and prefetching reliability. We analyze and evaluate the impact of prefetching reliability.

Kühn et al. [20] evaluated software prefetching on different CPUs and extensively analyzed the interplay between software prefetching and hardware. They found that the FB can quickly bottleneck software prefetching. To tackle this limitation, they suggest building hardware-conscious prefetching algorithms. We adopt the B^+-Tree benchmark introduced in their work, extend it with NUMA aware data placement for remote data placement on heterogeneous memory, and find that strong prefetching reliability causes poor performance for large B^+-Tree nodes.

Psaropoulos et al. [30] implemented binary search and B^+-Tree lookups with coroutines, group prefetching, and asynchronous memory access chaining in SAP HANA. The prefetching implementations outperform the baseline by over 2×. While coroutines show slightly higher performance overheads, their implementation effort is the lowest. Thus, the authors suggest using coroutines in real-world applications.

Jonathan et al. [15] evaluated different prefetching approaches, coroutine-based prefetching, group prefetching, and asynchronous memory access chaining, on hash tables, binary search, Masstrees, and Bw-trees. They experimentally analyzed the impact of data size, group size, compiler, and number of threads in a single CPU setup. Their coroutine implementations reach speedups between 1.4× and 8.2×.

Psaropoulos et al. [30] and Jonathan et al. [15] conducted experiments on Intel's Haswell and Broadwell, respectively. Jonathan et al. benchmarked their Masstree implementation on a dual-socket CPU. We evaluate prefetching approaches on multiple CPU architectures, in contrast to Psaropoulos et al. and Jonathan et al. Additionally, we run all experiments on multi-socket systems, which was only done in one experiment by Jonathan et al.

7 Conclusion

In this work, we analyze CPU implementations of software prefetching on seven CPUs using microbenchmarks for technical details and database workloads to determine performance implications.

We present a microbenchmark that analyzes software prefetching target caches and cache eviction based on different given localities. Only two of the seven analyzed CPUs share the same prefetching behavior and implementations w.r.t. the given locality. Differences for the most used localities `T0` and `NTA` are marginal, even

across x86 and ARM. We suggest using only `T0` and `NTA` for consistent and portable prefetching behavior.

We introduce two microbenchmarks that determine the prefetching reliability of CPUs and define the scenario in which prefetching reliability significantly impacts performance. Beyond this scenario, we find that the prefetching reliability only has a minor impact on performance. When issuing speculatively software prefetches, programmers should sort the issued prefetches by relevance. On CPUs with weak prefetching reliability, the first-issued prefetches are more likely to succeed. Programmers should limit prefetches on CPUs with strong prefetching probability to avoid unreasonable stalls caused by the CPU waiting for free FB slots.

We extend database workloads that use software prefetching with NUMA-aware data placement and thread scheduling to evaluate the impact of high memory access latency. We find that software prefetching can efficiently hide increased memory access latencies if the FBs are large enough to support enough outstanding memory requests. The required number of FBs increases with the memory intensity of the workload and the memory access latency. FB sizes are increasing, and technologies like CXL will bring even more heterogeneity and higher memory access latencies to servers [20, 33]. We therefore conclude that software prefetching will become even more attractive in the future.

Acknowledgments

We thank the anonymous reviewers for their helpful feedback and suggestions. This work was partially funded by the German Research Foundation (ref. 414984028), the European Union's Horizon 2020 research and innovation programme (ref. 957407), and by SAP.

References

[1] AMD. 2020. AMD64 Architecture Programmer's Manual Volume 1: Application Programming (Publication No. 24592). https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/40332.pdf

[2] AMD. 2024. AMD64 Architecture Programmer's Manual, Volume 3: General-Purpose and System Instructions (Publication No. 24594). https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/24594.pdf

[3] Magnus Bruce. 2023. Arm Neoverse V2 platform: Leadership Performance and Power Efficiency for Next-Generation Cloud Computing, ML and HPC Workloads. https://hc2023.hotchips.org/assets/program/conference/day1/CPU1/HC2023.Arm.MagnusBruce.v04.FINAL.pdf Accessed: 2025-01-20.

[4] Shimin Chen, Anastassia Ailamaki, Phillip B. Gibbons, and Todd C. Mowry. 2007. Improving hash join performance through prefetching. *ACM Trans. Database Syst.* 32, 3 (Aug. 2007), 17–es. https://doi.org/10.1145/1272743.1272747

[5] Shimin Chen, Phillip B. Gibbons, and Todd C. Mowry. 2001. Improving index performance through prefetching. In *Proceedings of the International Conference on Management of Data*, Vol. 30. ACM, Santa Barbara, CA, USA, 235–246. https://doi.org/10.1145/376284.375688

[6] John Cieslewicz, Jonathan Berry, Bruce Hendrickson, and Kenneth Ross. 2006. Realizing parallelism in database operations: Insights from a massively multithreaded architecture. In *Proceedings of the 2nd International Workshop on Data Management on New Hardware*, Vol. 2006. ACM, Chicago, Illinois, USA, 4. https://doi.org/10.1145/1140402.1140408

[7] Intel Corporation. 2024. Intel® 64 and IA-32 Architectures Optimization Reference Manual: Volume 1. https://www.intel.com/content/www/us/en/content-details/814198/intel-64-and-ia-32-architectures-optimization-reference-manual-volume-1.html

[8] Mohammad Ewais and Paul Chow. 2023. Disaggregated Memory in the Datacenter: A Survey. *IEEE Access* 11 (Feb. 2023), 20688–20712. https://doi.org/10.1109/ACCESS.2023.3250407 Conference Name: IEEE Access.

[9] Zhuhe Fang, Beilei Zheng, and Chuliang Weng. 2019. Interleaved multi-vectorizing. *Proceedings of the VLDB Endowment* 13 (Nov. 2019), 226–238. https://doi.org/10.14778/3368289.3368290

[10] Franz Färber, Norman May, Wolfgang Lehner, Philipp Große, Ingo Müller, Hannes Rauhe, and Jonathan Dees. 2012. The SAP HANA database - An architecture overview. *IEEE Data Eng. Bull.* 35 (March 2012), 28–33.

[11] Jim Gray, Prakash Sundaresan, Susanne Englert, Ken Baclawski, and Peter J. Weinberger. 1994. Quickly generating billion-record synthetic databases. In *Proceedings of the International Conference on Management of Data*, Vol. 23. ACM, Minneapolis, Minnesota, USA, 243–252. https://doi.org/10.1145/191843.191886

[12] Martin Grund, Jens Krüger, Hasso Plattner, Alexander Zeier, Philippe Cudre-Mauroux, and Samuel Madden. 2010. HYRISE: a main memory hybrid storage engine. In *Proceedings of the VLDB Endowment*, Vol. 4. VLDB Endowment, Seattle, WA, USA, 105–116. https://doi.org/10.14778/1921071.1921077

[13] Yongjun He, Jiacheng Lu, and Tianzheng Wang. 2020. CoroBase: Coroutine-Oriented Main-Memory Database Engine. *Proceedings of the VLDB Endowment* 14, 3 (2020), 431–444. https://doi.org/10.5555/3430915.3442440

[14] Intel. 2024. Intel® 64 and IA-32 Architectures Software Developer Manuals. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

[15] Christopher Jonathan, Umar Farooq Minhas, James Hunter, Justin Levandoski, and Gor Nishanov. 2018. Exploiting coroutines to attack the "killer nanoseconds". *Proceedings of the VLDB Endowment* 11 (July 2018), 1702–1714. https://doi.org/10.14778/3236187.3236216

[16] Alfons Kemper and Thomas Neumann. 2011. HyPer: A hybrid OLTP&OLAP main memory database system based on virtual memory snapshots. In *IEEE 27th International Conference on Data Engineering*. IEEE, Hannover, Germany, 195–206. https://doi.org/10.1109/ICDE.2011.5767867

[17] Yusuf Onur Koçberber, Babak Falsafi, and Boris Grot. 2015. Asynchronous Memory Access Chaining. *Proc. VLDB Endow.* 9, 4 (2015), 252–263. https://doi.org/10.14778/2856318.2856321

[18] Steve Kommrusch, Marcos Horro, Louis-Noël Pouchet, Gabriel Rodríguez, and Juan Touriño. 2021. Optimizing Coherence Traffic in Manycore Processors Using Closed-Form Caching/Home Agent Mappings. *IEEE Access* 9 (2021), 28930–28945. https://doi.org/10.1109/ACCESS.2021.3058280

[19] Yongkee Kwon. 2022. *Software Prefetching for Memory-level Parallelism.* Ph. D. Dissertation. The University of Texas at Austin. https://repositories.lib.utexas.edu/server/api/core/bitstreams/4ea2cf2e-d7e8-45df-a59f-aa334f477807/content

[20] Roland Kühn, Jan Mühlig, and Jens Teubner. 2024. How to Be Fast and Not Furious: Looking Under the Hood of CPU Cache Prefetching. In *Proceedings of the 20th International Workshop on Data Management on New Hardware (DaMoN '24)*. ACM, New York, NY, USA, 1–10. https://doi.org/10.1145/3662010.3663451

[21] Christoph Lameter. 2013. NUMA (Non-Uniform Memory Access): An Overview. *ACM Queue* 11, 7 (2013), 40. https://doi.org/10.1145/2508834.2513149

[22] Jaekyu Lee, Hyesoon Kim, and Richard W. Vuduc. 2012. When Prefetching Works, When It Doesn't, and Why. *ACM Trans. Archit. Code Optim.* 9, 1 (2012), 2:1–2:29. https://doi.org/10.1145/2133382.2133384

[23] Viktor Leis, Michael Haubenschild, and Thomas Neumann. 2019. Optimistic Lock Coupling: A Scalable and Efficient General-Purpose Synchronization Method. *IEEE Data Eng. Bull.* 42 (March 2019), 73–84. https://www.semanticscholar.org/paper/Optimistic-Lock-Coupling%3A-A-Scalable-and-Efficient-Leis-Haubenschild/b8b979480b98cd01ae283b34b15108d8c407c582

[24] Arm Limited. 2020. Arm A64 Instruction Set Architecture. https://developer.arm.com/documentation/ddi0596/2020-12/Base-Instructions/PRFM--immediate---Prefetch-Memory--immediate--

[25] Fujitsu Limited. [n. d.]. FUJITSU Processor A64FX Datasheet. https://www.fujitsu.com/downloads/SUPER/a64fx/a64fx_datasheet_en.pdf

[26] Fujitsu Limited. 2022. Fujitsu A64FX Microarchitecture Manual 1.8.1. https://github.com/fujitsu/A64FX/blob/master/doc/A64FX_Microarchitecture_Manual_en_1.8.1.pdf

[27] Gor Nishanov. 2018. Nano-coroutines to the Rescue! (Using Coroutines TS, of Course). https://www.youtube.com/watch?v=j9tlJAqMV7U

[28] NVIDIA. 2024. Grace Performance Tuning Guide — NVIDIA Grace Performance Tuning Guide. https://docs.nvidia.com/grace-perf-tuning-guide/index.html

[29] NVIDIA. 2024. NVIDIA Grace Hopper Superchip Data Sheet. https://resources.nvidia.com/en-us-grace-cpu/grace-hopper-superchip

[30] Georgios Psaropoulos, Thomas Legler, Norman May, and Anastasia Ailamaki. 2017. Interleaving with Coroutines: A Practical Approach for Robust Index Joins. *Proc. VLDB Endow.* 11, 2 (2017), 230–242. https://doi.org/10.14778/3149193.3149202

[31] Georgios Psaropoulos, Thomas Legler, Norman May, and Anastasia Ailamaki. 2019. Interleaving with coroutines: a systematic and practical approach to hide memory latency in index joins. *The VLDB Journal* 28 (Aug. 2019), 451–471. https://doi.org/10.1007/s00778-018-0533-6

[32] Michael Stonebraker and Ariel Weisberg. 2013. The VoltDB Main Memory DBMS. *IEEE Data Eng. Bull.* 36 (June 2013), 21–27. https://api.semanticscholar.org/CorpusID:6306329

[33] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. 2023. Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices. In *56th Annual IEEE/ACM International Symposium on Microarchitecture*. ACM, Toronto, ON, Canada, 105–121. https://doi.org/10.1145/3613424.3614256

[34] Midhul Vuppalapati, Saksham Agarwal, Henry Schuh, Baris Kasikci, Arvind Krishnamurthy, and Rachit Agarwal. 2024. Understanding the Host Network. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) *(ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 581–594. https://doi.org/10.1145/3651890.3672271

[35] Ziqi Wang, Andrew Pavlo, Hyeontaek Lim, Viktor Leis, Huanchen Zhang, Michael Kaminsky, and David G. Andersen. 2018. Building a Bw-Tree Takes More Than Just Buzz Words. In *Proceedings of the International Conference on Management of Data (SIGMOD '18)*. ACM, New York, NY, USA, 473–488. https://doi.org/10.1145/3183713.3196895

[36] Ying Wei, Yi Chieh Huang, Haiming Tang, Nithya Sankaran, Ish Chadha, Dai Dai, Olakanmi Oluwole, Vishnu Balan, and Edward Lee. 2023. 9.3 NVLink-C2C: A Coherent Off Package Chip-to-Chip Interconnect with 40Gbps/pin Single-ended Signaling. In *IEEE International Solid-State Circuits Conference*. IEEE, San Francisco, California, USA, 160–162. https://doi.org/10.1109/ISSCC42615.2023.10067395 ISSN: 2376-8606.

[37] Felix Werner, Marcel Weisgut, and Tilmann Rabl. 2025. Towards Memory Disaggregation via NVLink C2C: Benchmarking CPU-Requested GPU Memory Access. In *Proceedings of the 4th Workshop on Heterogeneous Composable and Disaggregated Systems (HCDS '25)*. Association for Computing Machinery, New York, NY, USA, 8–14. https://doi.org/10.1145/3723851.3723853